Nsuer Pro Clone — Project Blueprint

Planning document · October 17, 2026

1. Overview

Product: A web app (installable as a PWA) that helps university students plan course registration during the advising period. It is modeled on the idea of Nsuer Pro, an advising companion for North South University (NSU) students.

Core value: Registration is stressful because students must pick one section per course without time clashes, usually from a long, hard-to-read offered-courses list. The app makes that list searchable and readable, automatically finds clash-free timetables, and adds a CGPA calculator.

Four headline features

1. Offered courses search, with sections grouped by identical timing
2. Previous semesters' offered courses archive
3. Advising planner that generates clash-free section combinations, ranked by preference
4. CGPA calculator

Principle: Build an original product. Reproduce the feature ideas only. Do not copy the original's code, logo, copy text or visual design, and do not use NSU's logo or imply official affiliation.

2. Goals and non-goals

Goals

- A student can find a section and see its timing, faculty, room and seats in under 10 seconds.
- A student can pick 3–6 courses and receive ranked, clash-free timetables in under 2 seconds.
- Works well on a phone, installs like an app, and tolerates poor connectivity.
- Data for a new semester can be loaded by an admin in under 15 minutes.

Non-goals (for v1)

- Actually registering the student in the university system
- Storing student portal credentials, ever

- Scraping pages that require a student login
- Live seat tracking (show an “as of” timestamp instead)

3. Users and core flows

User	Goal
Student (anonymous)	Search courses, use the CGPA calculator, run the planner
Student (signed in, optional)	Save plans, store grade history, sync across devices
Admin (you or trusted moderators)	Import semester data, fix errors, publish a semester

Flow A — Plan a semester

1. Choose semester → add desired courses (e.g., 5 course codes).
2. Optionally set preferences: days off, earliest start, latest end, preferred or avoided faculty, maximum gap.
3. Press “Generate”. The solver returns the top N timetables.
4. Open a result, view the weekly grid, swap a section, save or export as an image.

Flow B — Find a section

1. Search by course code, title or faculty initials.
2. Filter by day, time window and seats available.
3. Results appear grouped: one row per distinct timing, listing all sections that share it.

Flow C — CGPA

1. Enter previous credits and CGPA, or add courses with grades one by one.
2. See semester GPA and cumulative CGPA update instantly.
3. Optional “what-if”: the GPA needed in remaining credits to reach a target CGPA.

4. Feature specifications

4.1 Offered courses (grouped by timing)

- Source: sections for the selected semester.
- Grouping key: sorted days + start time + end time. Example key: `MW|09:40-11:10`.

- Each group is a row. Inside the row, show a chip per section: course code, section number, faculty, seats, room.
- Filters: course code prefix, day pattern, time range, faculty, “has seats”.
- Sorting: by time group, then course code.
- Performance: load one semester at a time and filter on the client after the first fetch, since one semester is typically a few thousand rows at most.

4.2 Previous offered courses

- Same UI as 4.1 with a semester selector (Spring, Summer, Fall plus year).
- Purpose: students check which faculty and timings a course had in earlier terms to predict upcoming offerings.
- Optional extra: “Offered in” badge on a course page listing semesters where it ran.

4.3 CGPA calculator

- Grade scale (verify against the official NSU scale before shipping): A 4.0, A- 3.7, B+ 3.3, B 3.0, B- 2.7, C+ 2.3, C 2.0, C- 1.7, D+ 1.3, D 1.0, F 0.0.
- Formula: CGPA = Σ(grade point × credits) ÷ Σ(credits counted).
- Retakes: the policy decides whether the latest or the best attempt counts. Make this a configurable rule and confirm the policy.
- Non-GPA results (withdrawn, incomplete, pass/fail) are excluded from the sum.
- What-if solver: required GPA = (target × (doneCredits + remainingCredits) − currentPoints) ÷ remainingCredits.
- Everything runs in the browser. Saving to an account is optional.

4.4 Advising planner

- Input: list of course codes, semester, preferences.
- Output: ranked list of complete timetables (one section per course) with no time overlaps.
- Hard constraints: no clashes; optional blocked days or blocked time windows.
- Soft preferences (scored): free days, few early starts, few late finishes, small gaps, preferred faculty, seats available.
- If no clash-free combination exists, report which pair of courses conflicts most and suggest the least-bad alternative.
- If the course has a linked lab or recitation, treat the pair as one unit. Confirm how the source data represents this.

- "AI" layer (optional): an LLM turns plain-language wishes ("no Sunday classes, afternoons only") into the structured preference object. The solver itself needs no LLM.

4.5 Extras (after the MVP)

- Weekly timetable grid with drag-free section swapping
- Export as PNG or PDF
- Share a plan by link
- Dark mode
- Notifications when a full section gets seats (needs live data; later phase)

5. Data sourcing and ingestion

Data is the real difficulty. The code is routine.

Options, in order of preference

1. **Public source:** if the university publishes the offered-courses list on a public page or PDF, parse it with a script.
2. **Admin import:** an admin uploads a CSV or JSON each semester. This is the most reliable path for v1.
3. **Crowd-sourced corrections:** students flag wrong rows; an admin approves.

Avoid: scraping anything behind student login, or asking students for portal passwords. It is likely against the university's terms and creates serious security and trust problems.

Import pipeline

1. Admin uploads file → server parses rows.
2. Validate each row with Zod (required fields, time format, day codes, numeric seats).
3. Normalize: trim, uppercase course codes, convert times to minutes since midnight, parse day codes into arrays.
4. Show a preview with errors; admin confirms.
5. Upsert into the database in a single batch tagged with the semester and an `importedAt` timestamp.
6. Publish the semester so students can see it.

Day-code parsing: the source typically uses compact codes such as ST, MW, R, A (verify the real mapping). Map each letter to a day, then store `days: ["SUN","TUE"]`. Keep the original string in `rawDays` for debugging.

6. Data model

Store times as integers (minutes since midnight) so comparisons are trivial.

```
// Course — one per course code per catalog
interface Course {
  code: string;          // "CSE115"
  title: string;
  credits: number;
  department?: string;
}

// Section — one per offered section per semester
interface Section {
  semester: string;      // "Fall 2026"
  courseCode: string;
  sectionNo: number;
  faculty: string;       // initials or name
  days: Day[];           // ["SUN","TUE"]
  rawDays: string;       // "ST"
  start: number;         // 580 = 09:40
  end: number;           // 670 = 11:10
  room?: string;
  seats?: number;
  capacity?: number;
  type: "THEORY" | "LAB";
  linkedTo?: string;     // key of the paired lab/theory section
  importedAt: Date;
}

type Day = "SUN"|"MON"|"TUE"|"WED"|"THU"|"FRI"|"SAT";

interface Semester {
  name: string;          // "Fall 2026"
  published: boolean;
  importedAt: Date;
  sourceNote?: string;
}

// Optional, only with accounts
interface User {
  email: string;
  name?: string;
  createdAt: Date;
}
interface SavedPlan {
```

```
  userId: string;
  semester: string;
  sectionKeys: string[]; // ["CSE115-3", ...]
  label?: string;
  createdAt: Date;
}
interface GradeRecord {
  userId: string;
  courseCode: string;
  credits: number;
  grade: string;
  semester: string;
}
```

Indexes

- Section: unique on `(semester, courseCode, sectionNo)`
- Section: `(semester, courseCode)` for planner lookups
- Text index on course code, title and faculty for search
- Course: unique on `code`

7. Algorithms

7.1 Grouping sections by timing

```
const key = (s: Section) =>
`${[...s.days].sort().join("")}|${s.start}-${s.end}`;
const groups = new Map<string, Section[]>();
for (const s of sections) {
  const k = key(s);
  (groups.get(k) ?? groups.set(k, []).get(k)!).push(s);
}
```

7.2 Clash detection

Two sections clash if they share a day and their time ranges overlap:

```
const clash = (a: Section, b: Section) =>
  a.days.some(d => b.days.includes(d)) && a.start < b.end && b.start < a.end;
```

For speed, precompute a bitmask per section: one bit per 10-minute slot per day. Two sections clash when `(maskA & maskB) !== 0n`. Use BigInt or split into several 32-bit integers.

7.3 Planner (backtracking with pruning)

```
function solve(courses: Section[][], limit = 500) {
  const out: Section[][] = [];
  const picked: Section[] = [];
  (function go(i: number) {
    if (out.length >= limit) return;
    if (i === courses.length) { out.push([...picked]); return; }
    for (const s of courses[i]) {
      if (picked.every(p => !clash(p, s))) {
        picked.push(s); go(i + 1); picked.pop();
      }
    }
  })(0);
  return out;
}
```

Optimizations

- Order courses by fewest sections first, so dead ends are found early.
- Drop sections that violate hard constraints (blocked days or times) before the search.
- Stop at a result limit, then score and sort only those.
- Worst case with 6 courses and 10 sections each is 1,000,000 combinations. Pruning keeps real inputs far smaller. For heavy cases, run the solver in a Web Worker so the UI never freezes.

7.4 Ranking

Score each timetable with weighted terms, then sort descending:

```
score = w1 × freeDays
      − w2 × totalGapMinutes
      − w3 × classesBefore(earliestPreferred)
      − w4 × classesAfter(latestPreferred)
      + w5 × preferredFacultyCount
      − w6 × avoidedFacultyCount
```

Expose two or three preset profiles (“Compact days”, “Late starts”, “Fewest gaps”) so most students never touch the weights.

8. System architecture and tech stack

Layer	Choice	Notes
Framework	Next.js 15 (App Router) + TypeScript	Server Components for data pages, client components for planner and calculator
Styling	Tailwind CSS v4	Design tokens for light and dark themes
Database	MongoDB Atlas + Mongoose	A free tier is enough at the start
Validation	Zod	Shared by import pipeline and API
Auth (optional)	Auth.js	Email magic link or Google; skip for MVP
State	React state, Zustand for planner	Keep it simple
Solver runtime	Web Worker	Keeps the UI responsive
PWA	Serwist (service worker) + web manifest	Cache the app shell and the last-viewed semester
Export	html-to-image or canvas	Timetable PNG
Hosting	Vercel	Preview deploys per branch
Testing	Vitest, Playwright	Solver and parsers get the most tests

Data flow: Admin CSV → import API → MongoDB → Route Handlers/Server Components → client cache → planner worker.

Caching: semester data changes rarely, so serve it with long cache headers and revalidate on import (Next.js `revalidateTag`).

9. Pages and routes

Route	Purpose
`/`	Landing: what the app does, links to the four tools
`/dashboard`	Tool hub
`/dashboard/offered-courses`	Search and grouped view for the current semester

Route	Purpose
`/dashboard/previous-courses`	Same, with semester selector
`/dashboard/advising-planner`	Course picker, preferences, results, timetable grid
`/dashboard/cgpa-calculator`	Calculator and what-if
`/dashboard/plans`	Saved plans (signed in)
`/admin`	Import, preview, publish, edit rows (admin only)
`/privacy-policy`, `/terms`	Legal pages

10. API design

Method and path	Purpose
`GET /api/semesters`	List published semesters
`GET /api/sections?semester=&q=&day=&faculty=&seats=`	Filtered sections
`GET /api/sections/grouped?semester=`	Sections pre-grouped by timing
`GET /api/courses?q=`	Course autocomplete for the planner
`POST /api/planner/solve`	Server-side solve (fallback if the worker is unavailable)
`POST /api/plans` / `GET /api/plans`	Save and list plans (auth)
`POST /api/admin/import`	Upload and validate a semester file (admin)
`POST /api/admin/publish`	Publish a semester (admin)

Protect `/api/admin/*` with role checks and rate limits. Return typed errors with row numbers for import problems.

11. Project structure

```
src/
  app/
    (marketing)/page.tsx
    dashboard/
```

```
      offered-courses/page.tsx
      previous-courses/page.tsx
      advising-planner/page.tsx
      cgpa-calculator/page.tsx
      plans/page.tsx
    admin/page.tsx
    api/
      sections/route.ts
      planner/solve/route.ts
      admin/import/route.ts
  components/
    timetable-grid/
    section-chip/
    course-picker/
    ui/
  lib/
    db.ts
    models/ (course.ts, section.ts, semester.ts)
    schemas/ (zod)
    time.ts          // minutes <-> "HH:MM"
    days.ts          // day-code parser
    planner/
      clash.ts
      solve.ts
      score.ts
      worker.ts
    cgpa/
      scale.ts
      calc.ts
  tests/
    planner.test.ts
    days.test.ts
    cgpa.test.ts
public/
  manifest.webmanifest
  icons/
```

12. UI and UX guidelines

- Mobile first. Most students will use this on a phone during a crowded registration window.
- Timetable grid: days as columns, time as rows, color per course, clash highlighted in red if the user manually forces a swap.
- Fast feedback: search as you type with a short debounce; the planner shows progress.

- Empty and error states written in plain language (“No clash-free combination. Courses X and Y overlap in every section.”).
- Accessibility: keyboard navigable, sufficient contrast in both themes, no color-only meaning.
- Original branding: your own name, logo and palette.

13. Security, privacy and legal

- **No portal credentials.** Never ask for or store them.
- **Minimal data.** Anonymous use should need no account. For accounts, store only email, plans and optional grades.
- **Admin protection:** role-based access, rate limiting, audit log of imports.
- **Input validation** on every endpoint; sanitize anything rendered back.
- **Privacy policy** explaining what is stored and how to delete it; provide an account-deletion option.
- **Disclaimer:** unofficial tool; data may contain errors; students must confirm in the official system.
- **Intellectual property:** do not copy the original site’s code, text, images or logo. Do not use the university’s marks without permission.
- **Data rights:** make sure republishing the course list is acceptable under the university’s terms.

14. Testing, deployment and operations

- **Unit tests:** day-code parser, time conversion, clash detection, grouping, CGPA calculation, scoring. The solver deserves property-style tests (every returned plan has zero clashes).
- **Fixtures:** keep a small, realistic sample semester file for tests and demos.
- **E2E (Playwright):** import → publish → search → plan → export.
- **Environments:** local, preview, production, each with its own database.
- **Environment variables:** `MONGODB_URI`, `AUTH_SECRET`, `ADMIN_EMAILS`, plus an LLM API key only if the natural-language layer is added.
- **Monitoring:** error tracking (for example Sentry) and basic analytics focused on feature usage, without personal data.
- **Backups:** scheduled database backups before each semester import.

15. Roadmap

Phase	Scope	Rough effort
0. Setup	Repo, Tailwind v4, MongoDB connection, CI, design tokens	1–2 days
1. CGPA calculator	Grade scale, calculation, retakes, what-if, tests	2–3 days
2. Data layer	Schemas, day/time parsers, admin CSV import with preview	4–6 days
3. Course search	Offered courses, grouped view, filters, previous semesters	4–5 days
4. Planner	Solver, worker, preferences, ranking, timetable grid	6–8 days
5. Polish	PWA, dark mode, export, empty states, accessibility	3–5 days
6. Accounts (optional)	Auth, saved plans, grade history	4–6 days
7. Launch	Privacy policy, disclaimer, real-semester data load, beta with a few students	2–3 days

A solo developer comfortable with Next.js can reach a usable MVP (phases 0–4) in roughly 3–4 weeks of part-time work. Estimates assume the semester data is obtainable in a clean format.

16. Risks and mitigations

Risk	Mitigation
Course data is hard to obtain or format changes each semester	Build the importer around a flexible column-mapping step; keep admin editing tools
Wrong data causes a student to plan around a section that does not exist	“Data as of” timestamp, report-an-error button, visible disclaimer
Solver is slow for many courses	Fewest-sections-first ordering, bitmasks, Web Worker, result limit
University objects to republishing data	Ask for permission, offer to link officially, keep the tool clearly unofficial
Registration-day traffic spikes	Static caching, CDN, read-only endpoints, cheap database queries

Risk	Mitigation
Scope creep	Ship phases 0–4 first; everything else is optional

17. Things to verify before building

1. The official grade scale and the retake rule for CGPA.
2. How the university publishes offered courses (public page, PDF, login-only) and the exact column layout.
3. The real day-code mapping and the standard class time slots.
4. How lab or recitation sections are linked to theory sections in the data.
5. Whether republishing the course list is allowed, and whether the university would endorse the tool.
6. Your monetization plan, if any (free with donations, ads, or paid extras), and the payment options that work for your users.